check).

#4 We can only perform a right rotation on a left child: so, if `x` is `y`'s right child, there is an error.

#5 Use a convenience variable for `y`'s parent.

#6 This time we don't know if `p` is `null` or not, so we need to check.

#7 Once we know `p` is not `null`, we still don't know if `y` is its left or right children.

#8 If there is a parent for `y`, we need to update it, replacing `y` with `x`.

#9 If, instead, `p` is `null`, it means that `y` was the treap's root, and so we need to update the tree making `x` its new root.

#10 Now that a reference to `x` is stored (either as `p`'s child or as the new root), we can update `y`'s left subtree, that will now point to `x`'s former right subtree.

#11 Finally, we reconnect `y` to the tree by setting it as `x`'s new left child.

`Y`'s left subtree, instead, is empty because we disconnected `X` and moved it: we can then move `X`'s right subtree and assign it to `Y`'s left child (line #10), as shown in the lower-left section of figure 3.5. This certainly doesn't violate the key ordering, because (assuming there was no violation *before* the rotation) `key[Y]>=key[Y.left]` and `key[Y]>=key[Y.left.right]`: in other words, since `X` was the left child of node `Y`, then the right subtree of node `X` is still in `Y`'s left subtree, and all keys in a node's left subtree are smaller (or at most equal) to the node's own key (you can also use figure 3.2 as a reference).